MLOps Experimental Learning Assignment: End-to-End ML Model Development, CI/CD, and Production Deployment Experimental Learning

Machine Learning Operations (MLOps) (AIMLCZG523) | Total marks: 50

Student: SHREYES RAM V. | BITS ID: 2025AE05177

Repository: https://github.com/ShreyStar02/MLOps_Assignment1_Shrey

Video recording: Not supplied

Objective

Design, develop, and deploy a scalable, reproducible machine learning solution using modern MLOps best practices, including practical automation, experiment tracking, CI/CD, containerization, deployment, and monitoring.

Contents

1. Data acquisition and exploratory analysis

2. Feature engineering and model development

3. Experiment tracking

4. Model packaging and reproducibility

5. CI/CD and automated testing

6. Containerized API

7. Kubernetes deployment

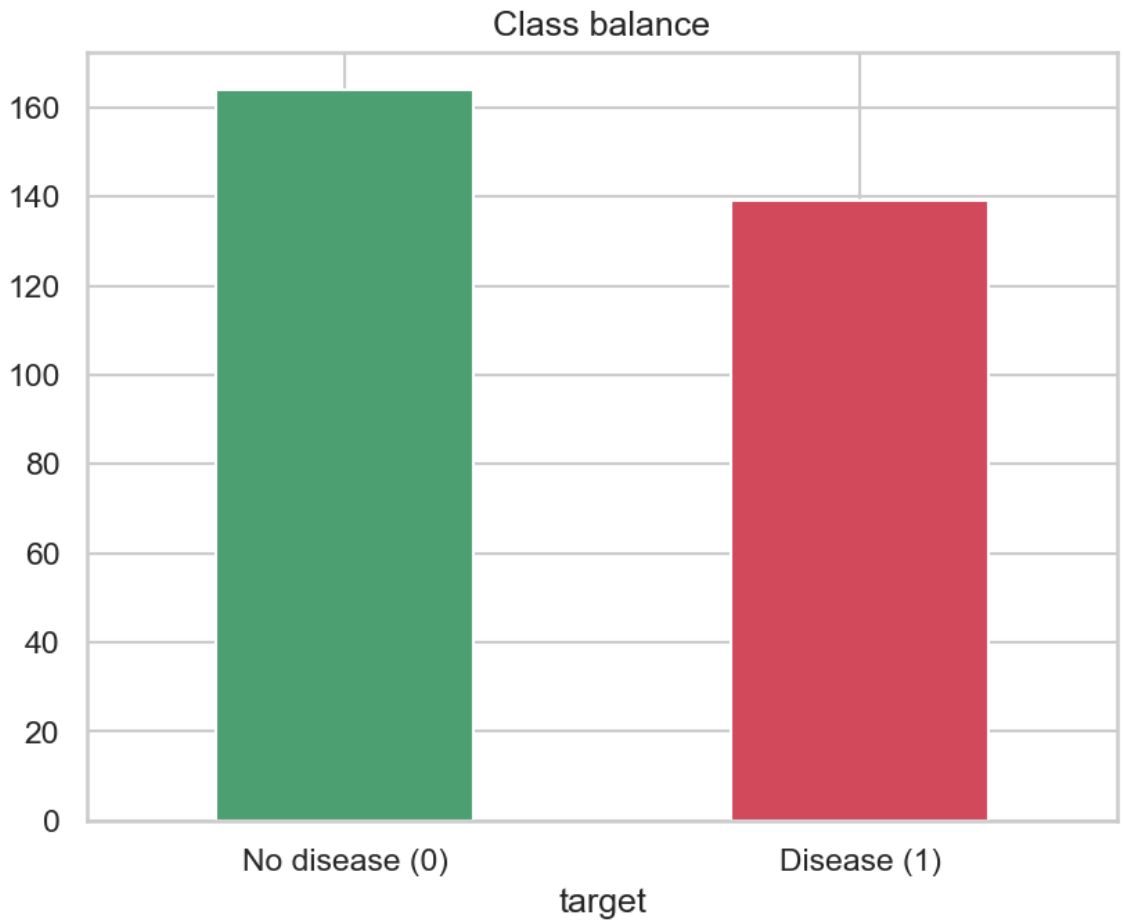
8. Monitoring and logging

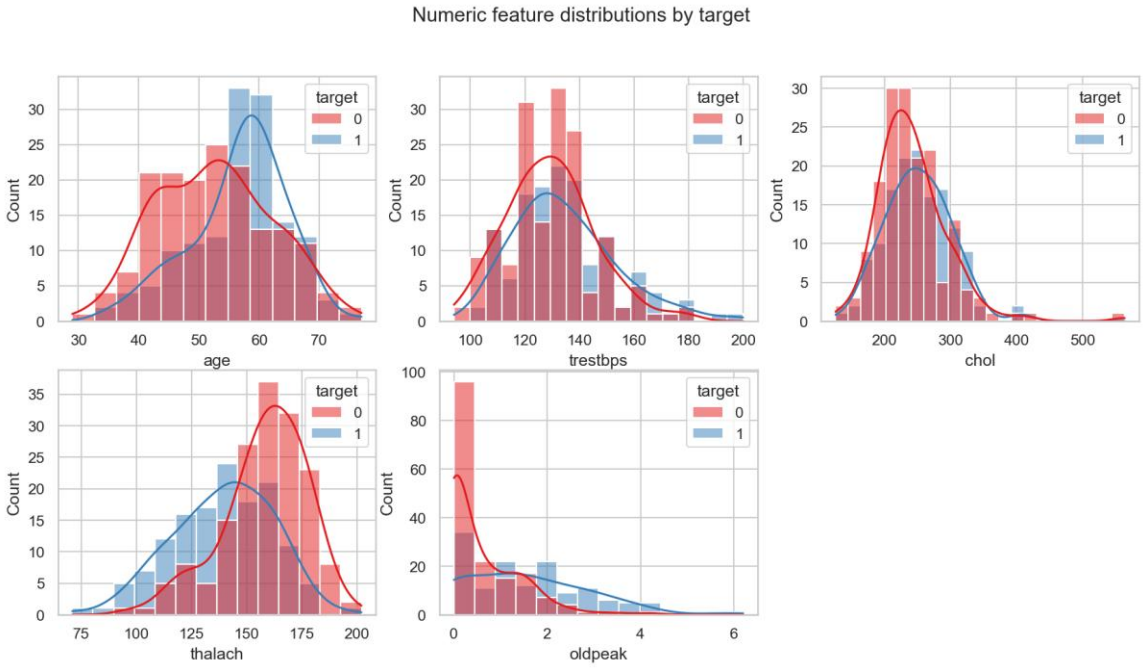
9. Architecture

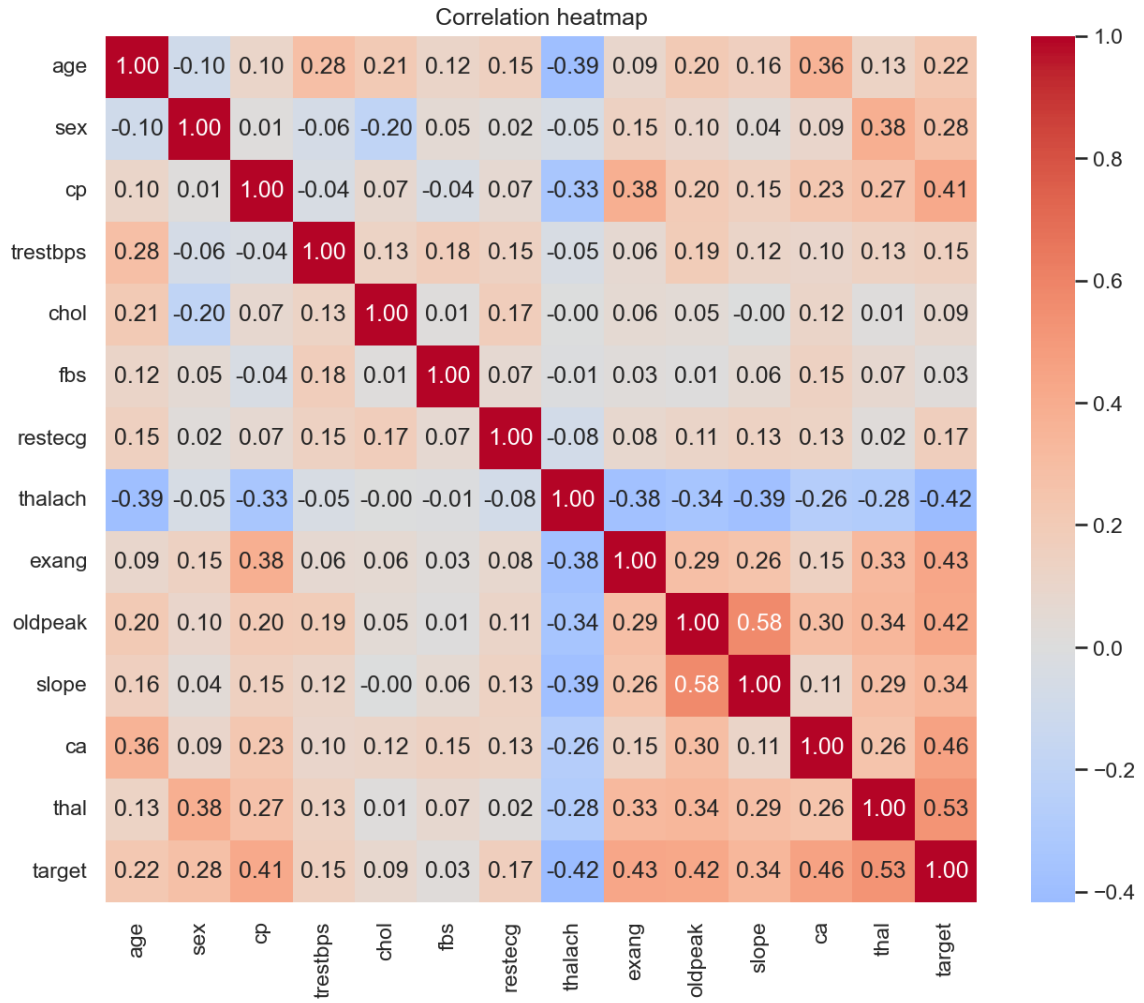
10. Local access and submission status

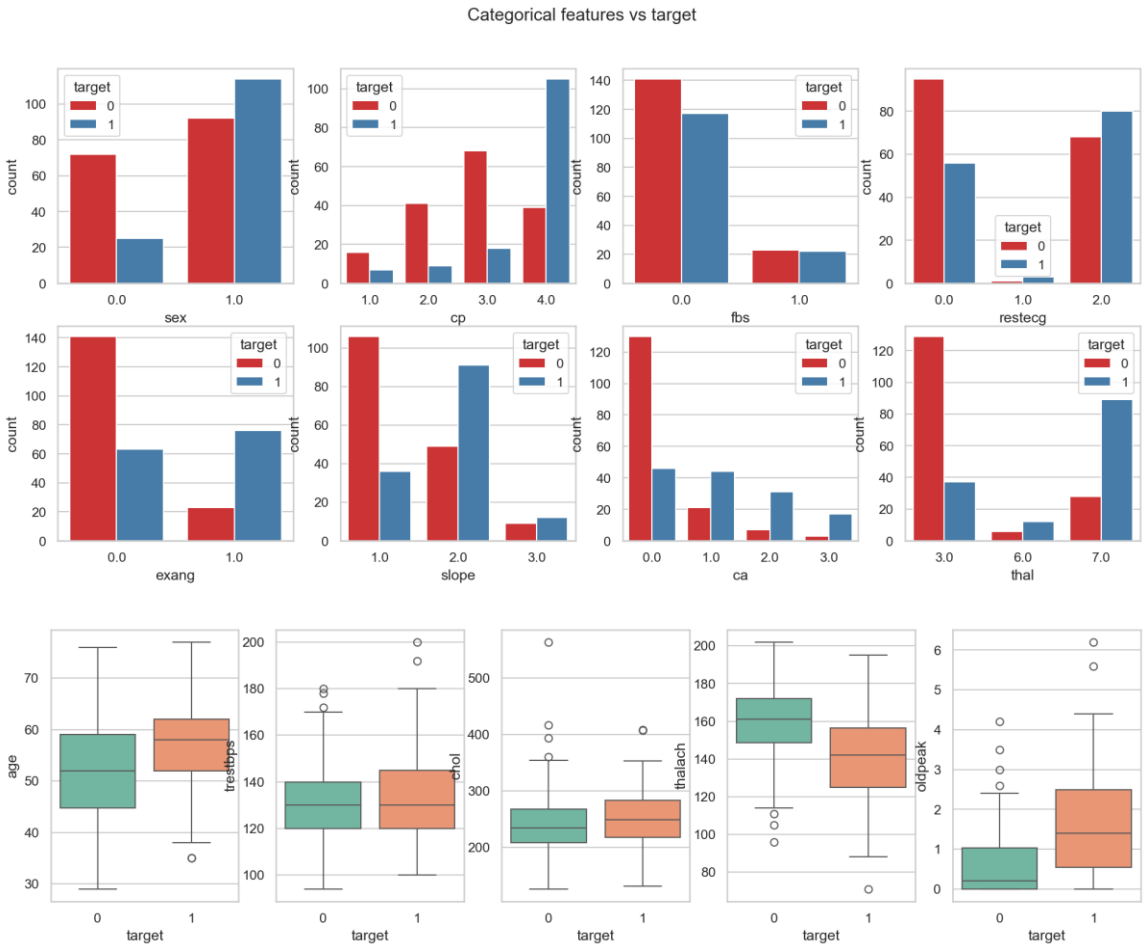
1. Data acquisition and exploratory analysis

The UCI Cleveland heart disease data contains 303 records. The target counts are 164 without disease and 139 with disease. `ca` has 4 missing values and `thal` has 2. The cleaning script preserves these records so the preprocessing pipeline can impute missing features.




Numeric feature distributions by target
target
0
1
Count
age
trestbps
chol
thalach
oldpeak

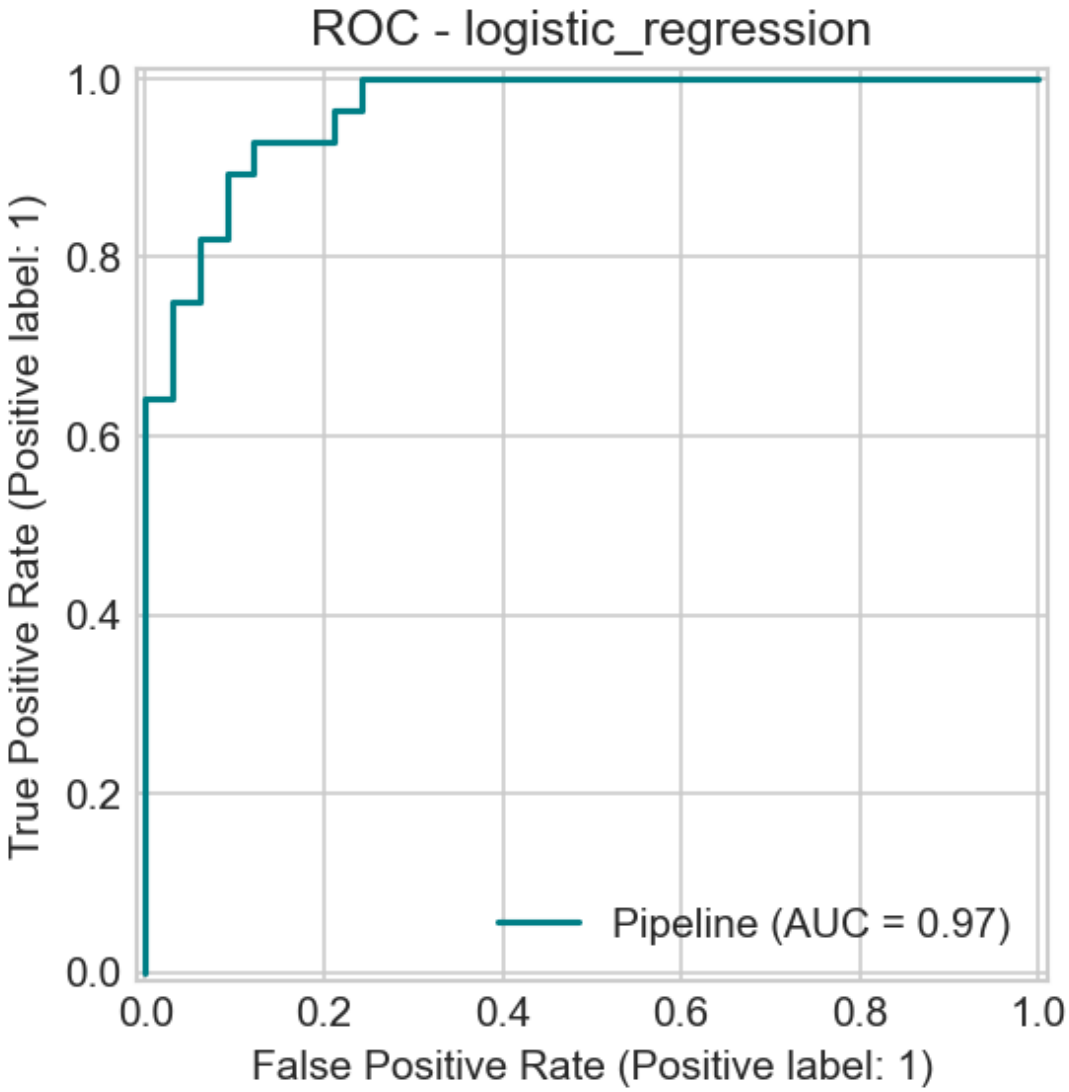

Correlation heatmap
age 1.00 -0.10 0.10 0.28 0.21 0.12 0.15 -0.39 0.09 0.20 0.16 0.36 0.13 0.22
sex -0.10 1.00 0.01 -0.06 -0.20 0.05 0.02 -0.05 0.15 0.10 0.04 0.09 0.38 0.28
cp 0.10 0.01 1.00 -0.04 0.07 -0.04 0.07 -0.33 0.38 0.20 0.15 0.23 0.27 0.41
trestbps 0.28 -0.06 -0.04 1.00 0.13 0.18 0.15 -0.05 0.06 0.19 0.12 0.10 0.13 0.15
chol 0.21 -0.20 0.07 0.13 1.00 0.01 0.17 -0.00 0.06 0.05 -0.00 0.12 0.01 0.09
fbs 0.12 0.05 -0.04 0.18 0.01 1.00 0.07 -0.01 0.03 0.01 0.06 0.15 0.07 0.03
restecg 0.15 0.02 0.07 0.15 0.17 0.07 1.00 -0.08 0.08 0.11 0.13 0.13 0.02 0.17
thalach -0.39 -0.05 -0.33 -0.05 -0.00 -0.01 -0.08 1.00 -0.38 -0.34 -0.39 -0.26 -0.28 -0.42
exang 0.09 0.15 0.38 0.06 0.06 0.03 0.08 -0.38 1.00 0.29 0.26 0.15 0.33 0.43
oldpeak 0.20 0.10 0.20 0.19 0.05 0.01 0.11 -0.34 0.29 1.00 0.58 0.30 0.34 0.42
slope 0.16 0.04 0.15 0.12 -0.00 0.06 0.13 -0.39 0.26 0.58 1.00 0.11 0.29 0.34
ca 0.36 0.09 0.23 0.10 0.12 0.15 0.13 -0.26 0.15 0.30 0.11 1.00 0.26 0.46
thal 0.13 0.38 0.27 0.13 0.01 0.07 0.02 -0.28 0.33 0.34 0.29 0.26 1.00 0.53
target 0.22 0.28 0.41 0.15 0.09 0.03 0.17 -0.42 0.43 0.42 0.34 0.46 0.53 1.00
age sex cp trestbps chol fbs restecg thalach exang oldpeak slope ca thal target
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4

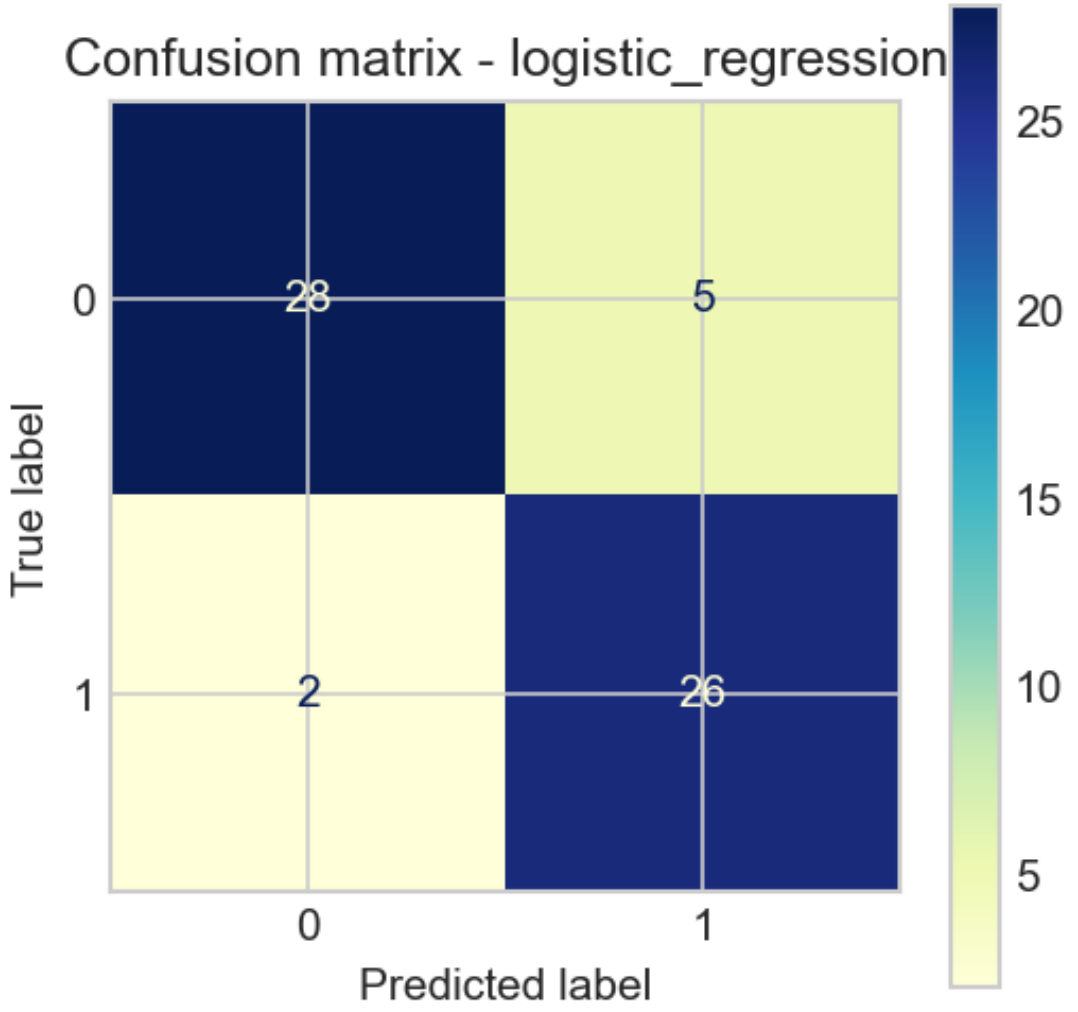


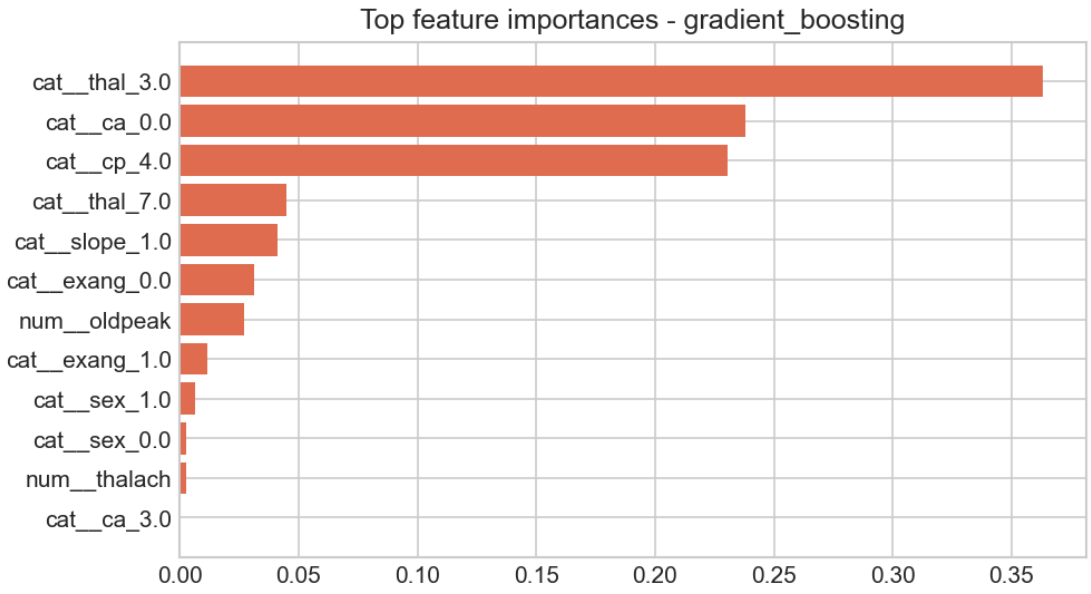
2. Feature engineering and model development

The pipeline engineers an age bucket, median-imputes and scales numeric features, and most-frequent-imputes and one-hot encodes categorical features. Logistic regression, random forest, and gradient boosting were tuned with GridSearchCV and evaluated with stratified five-fold CV.

Model	CV ROC-AUC	Test ROC-AUC	Test accuracy	Test F1
Logistic Regression	0.901	0.965	0.885	0.881
Random Forest	0.901	0.951	0.869	0.867
Gradient Boosting	0.897	0.943	0.869	0.857

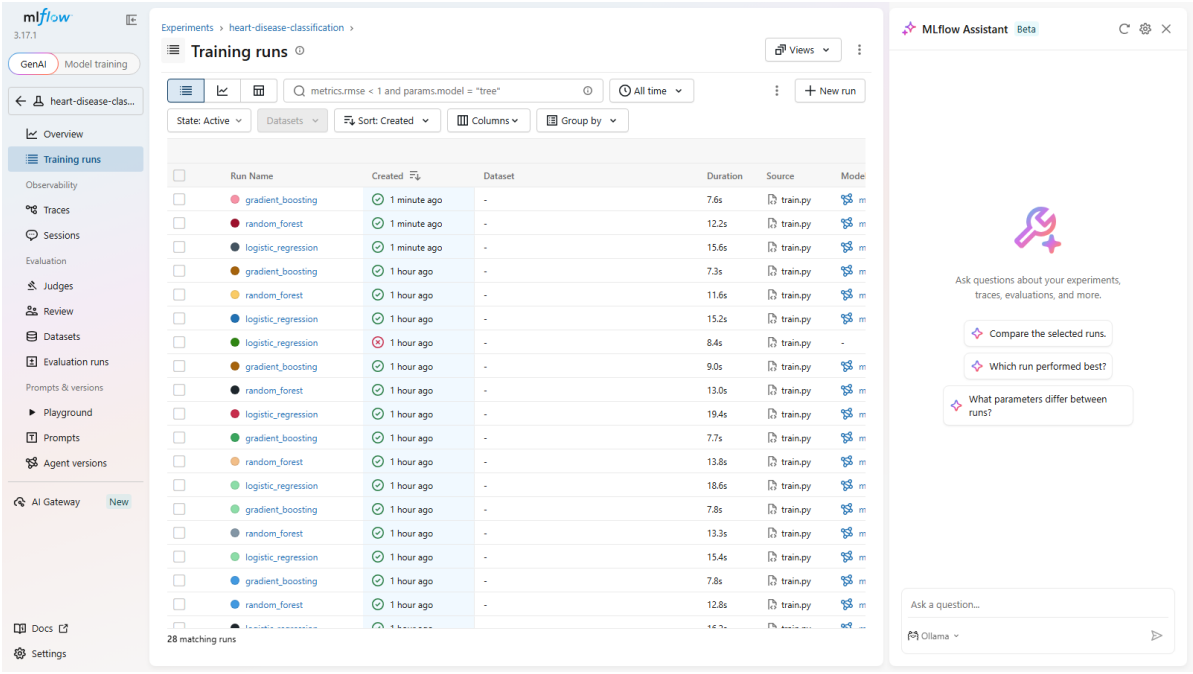

ROC - logistic_regression
True Positive Rate (Positive label: 1)
False Positive Rate (Positive label: 1)
0.0
0.2
0.4
0.6
0.8
1.0
Pipeline (AUC = 0.97)

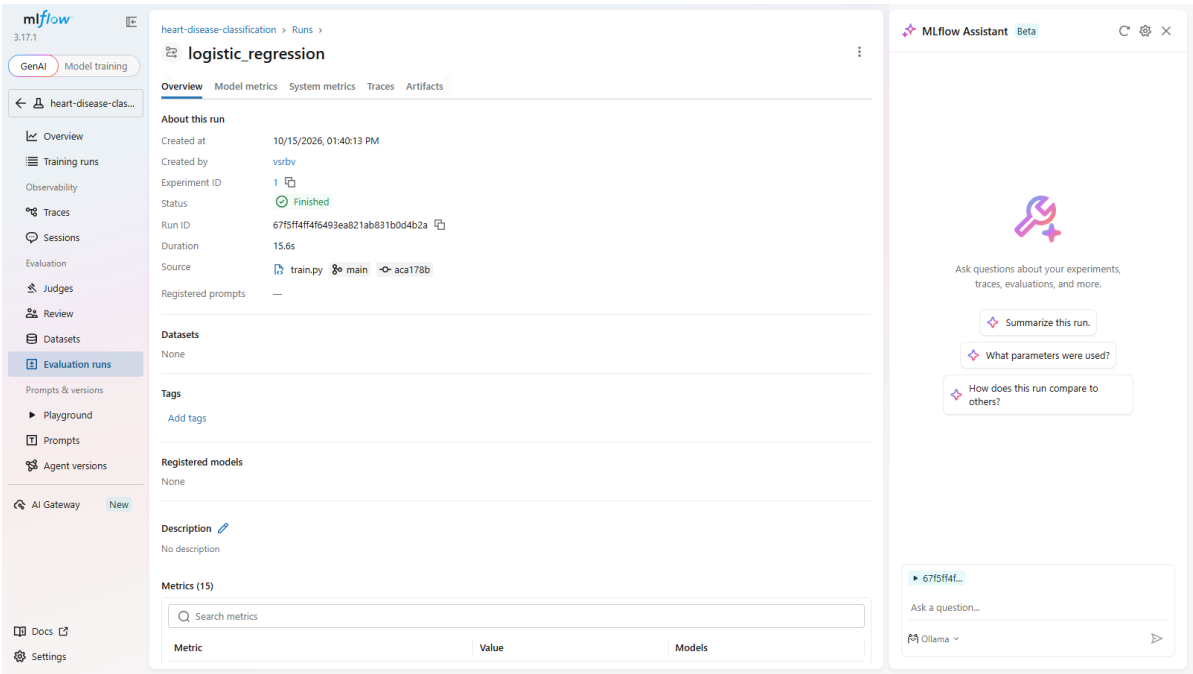

Confusion matrix - logistic_regression
True label
Predicted label
0
1
0
1
28
5
2
26
25
20
15
10
5


Top feature importances - gradient_boosting
cat__thal_3.0
cat__ca_0.0
cat__cp_4.0
cat__thal_7.0
cat__slope_1.0
cat__exang_0.0
num__oldpeak
cat__exang_1.0
cat__sex_1.0
cat__sex_0.0
num__thalach
cat__ca_3.0
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35

3. Experiment tracking

MLflow stores each candidate run, its parameters and CV/holdout metrics, plots, and model artifact in the local SQLite tracking store.





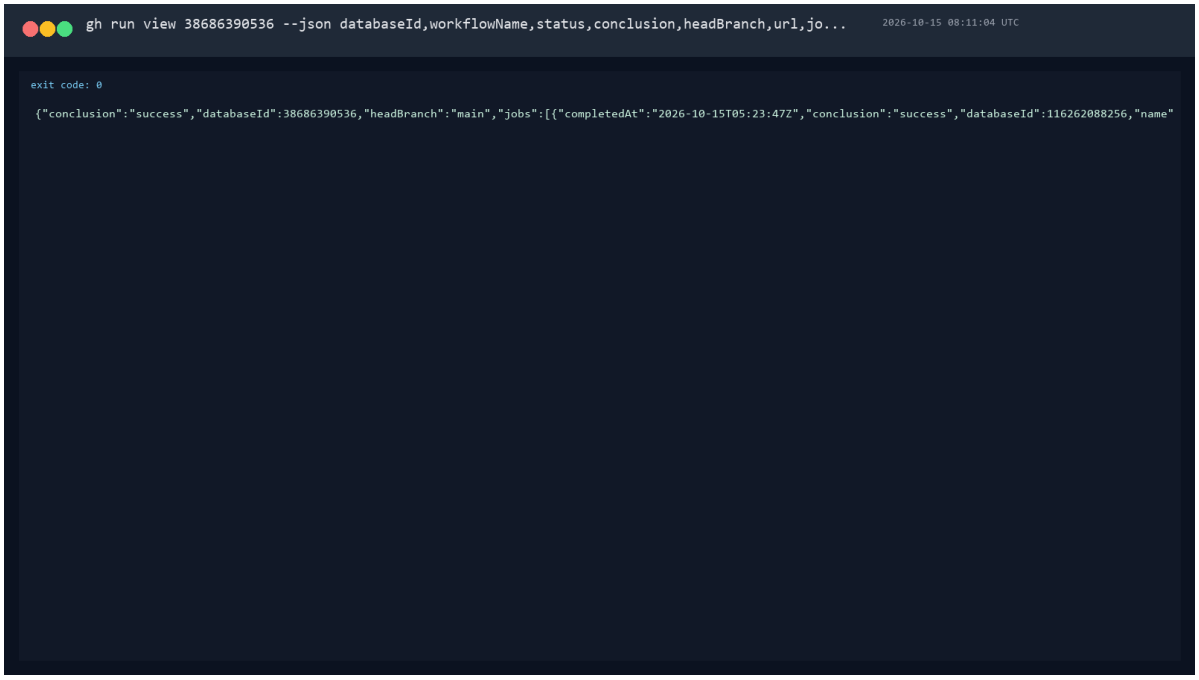
4. Model packaging and reproducibility

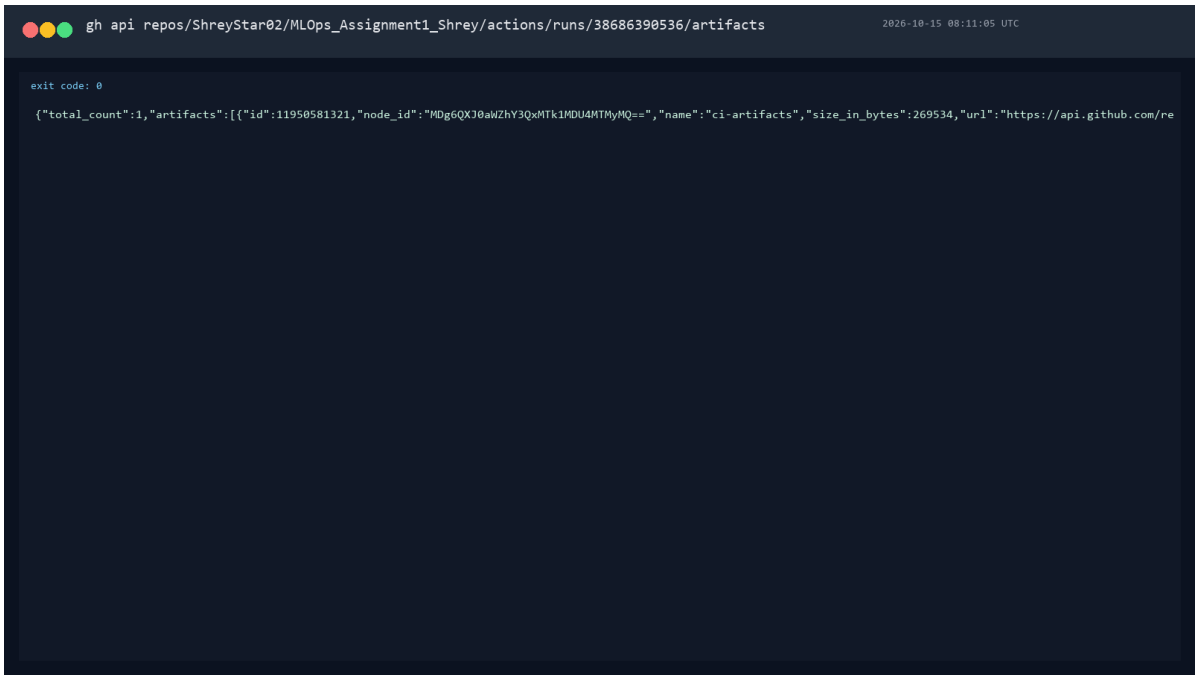
The selected logistic_regression pipeline uses scikit-learn 1.7.2 and is serialized to models/model.joblib. models/metadata.json records the selected parameters, feature

names, metrics, and MLflow run ID. Training and serving share the same preprocessing pipeline.

5. CI/CD and automated testing

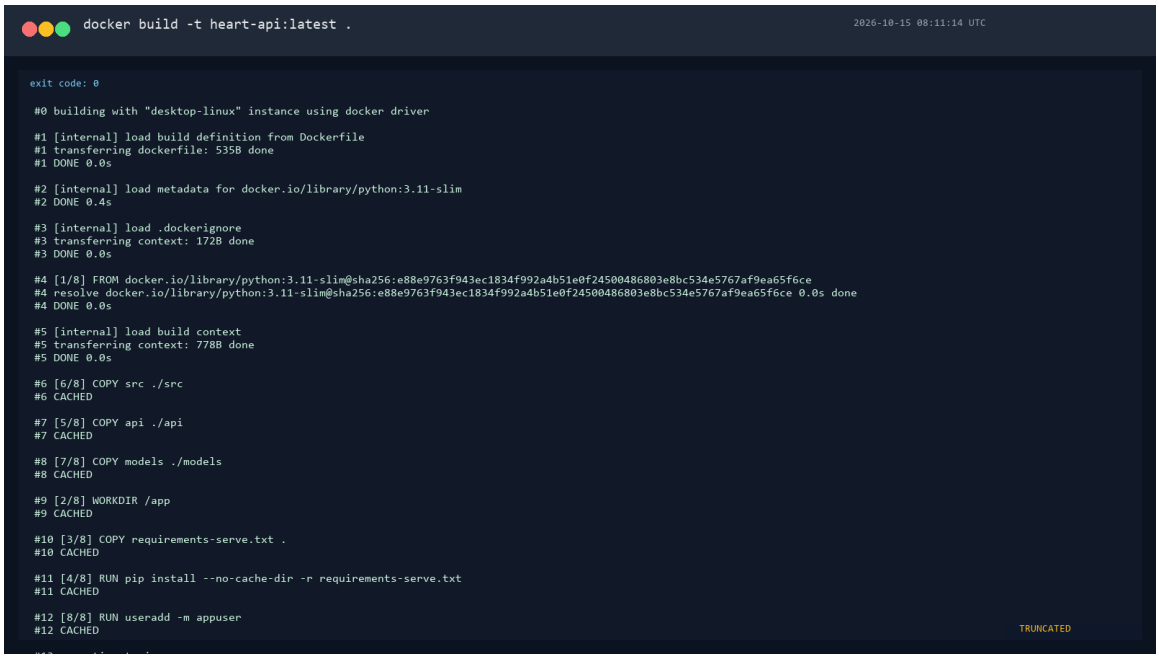
GitHub Actions runs linting, unit tests with coverage, data download, training, Docker build and API smoke tests, then uploads model and training artifacts. The final local run passed all 14 tests and Flake8.

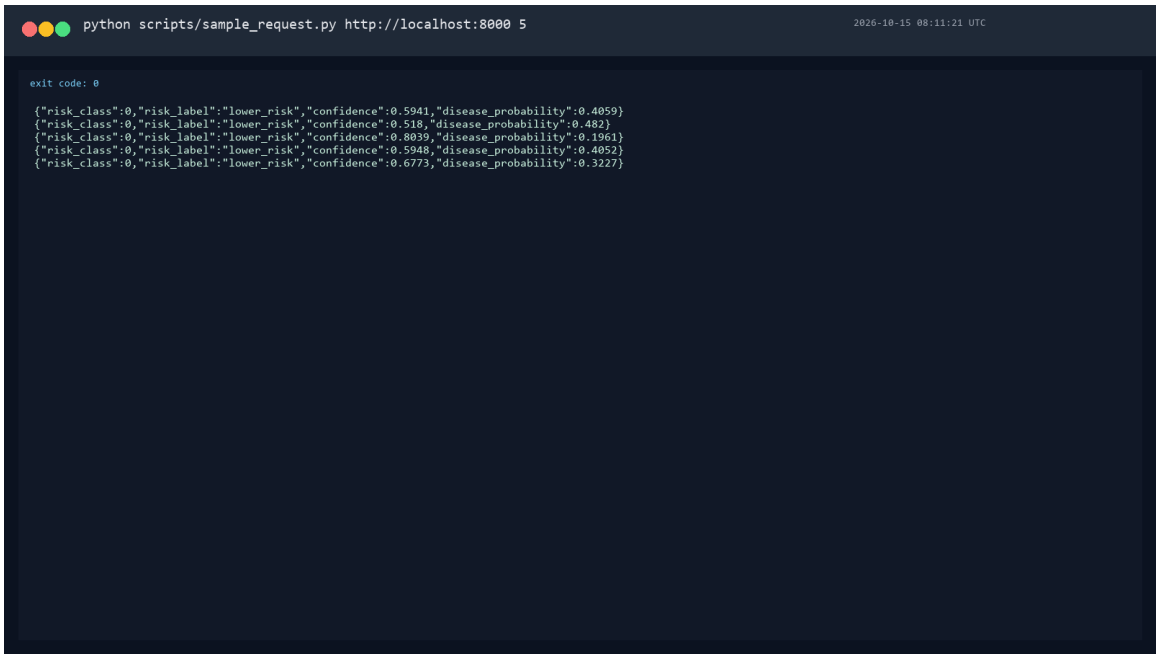


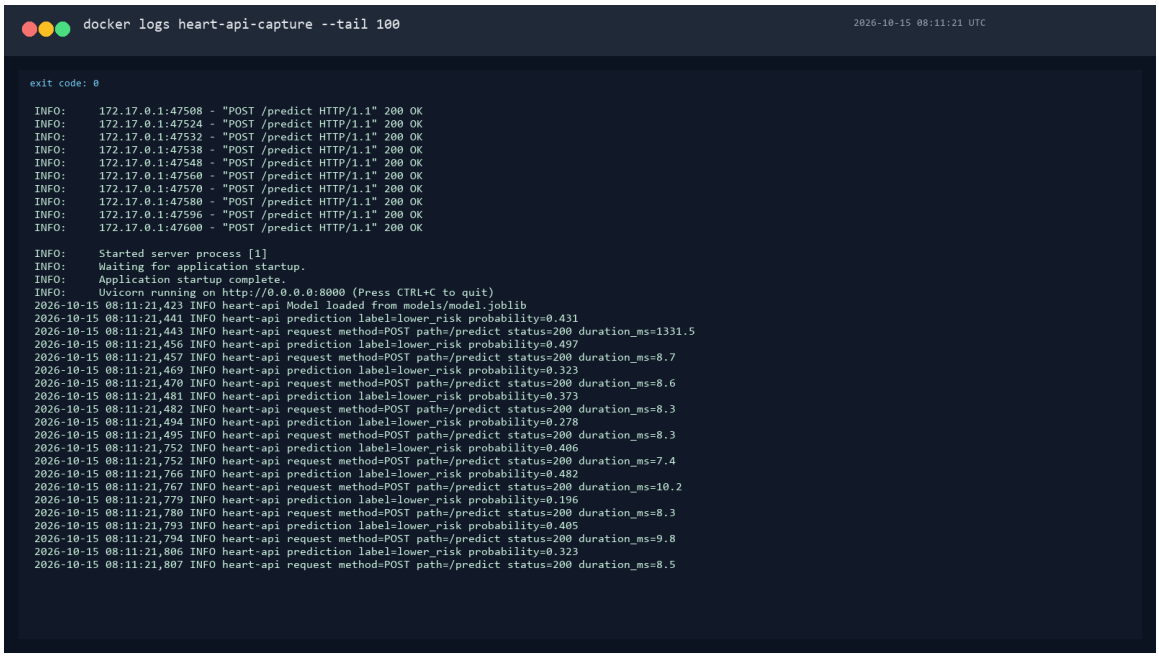


6. Containerized API

FastAPI accepts patient features at POST /predict and returns risk class, risk label, confidence, and disease probability. GET /health, GET /model-info, GET /metrics, and /docs support health checks, metadata, metrics, and API exploration. Request logging records status and latency.

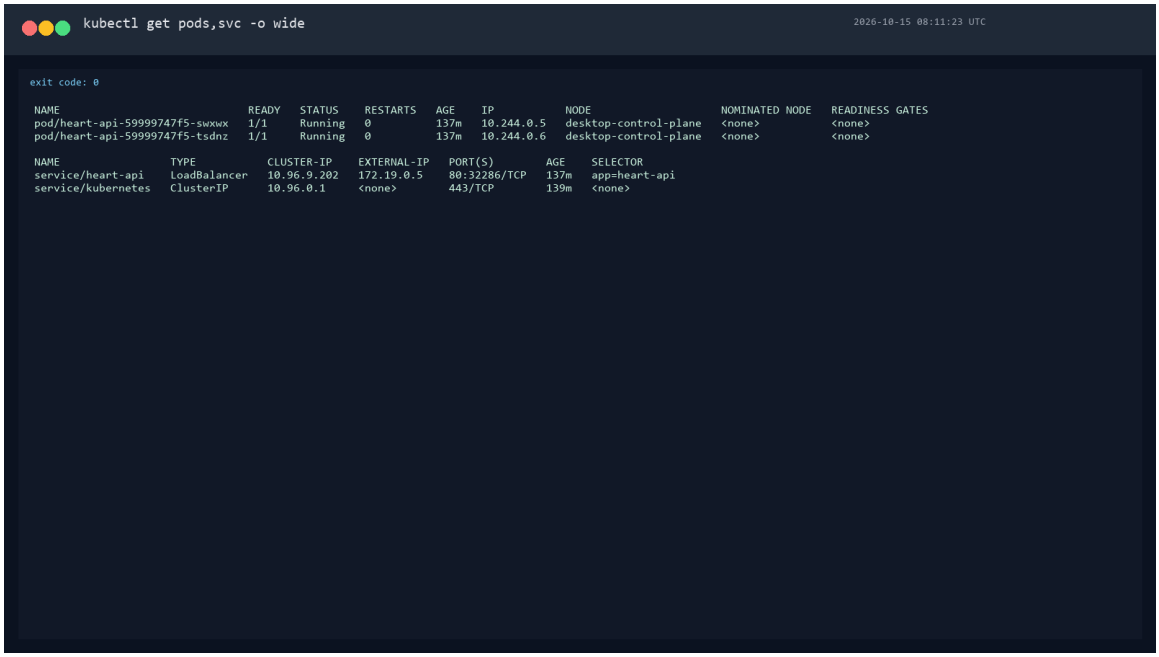


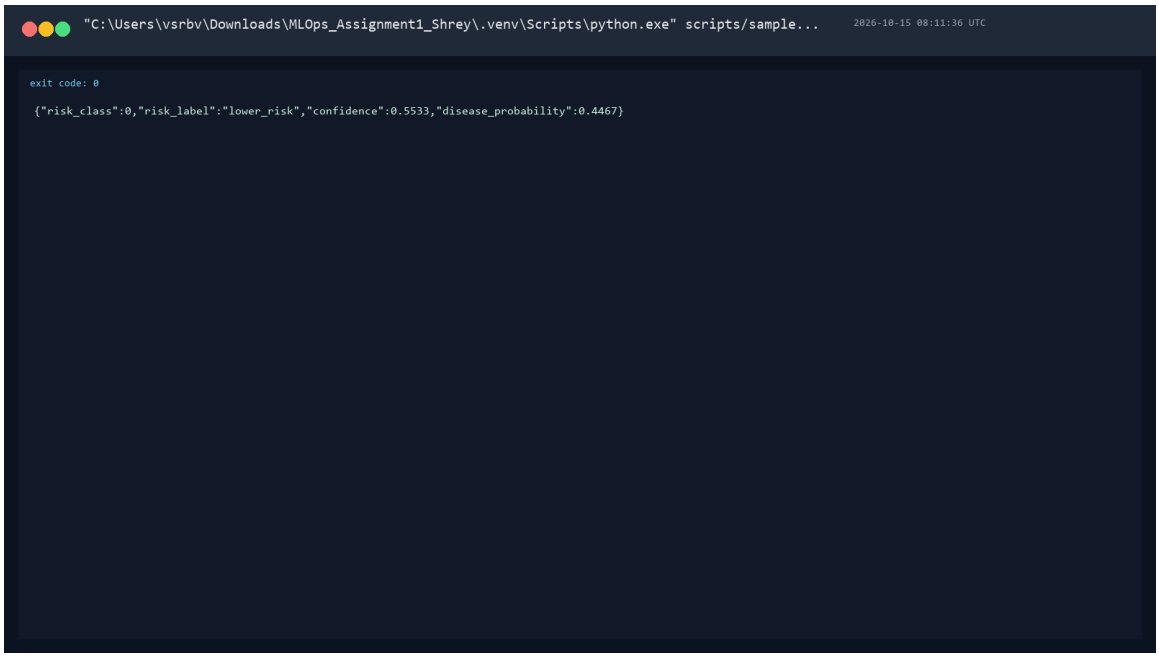




7. Kubernetes deployment

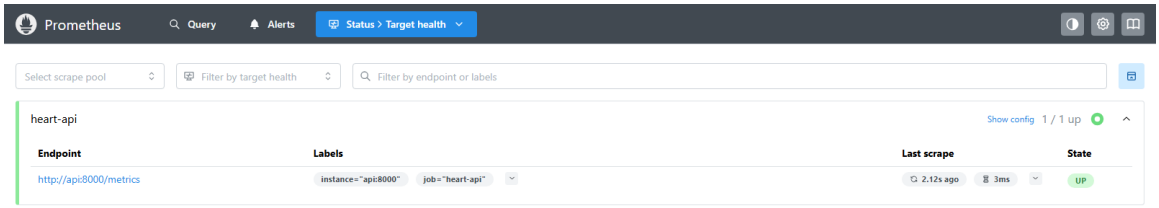
The Kubernetes deployment runs two replicas behind a LoadBalancer service. Readiness and liveness probes call /health. Both pods reached Ready and a sample prediction succeeded through a local port-forward.

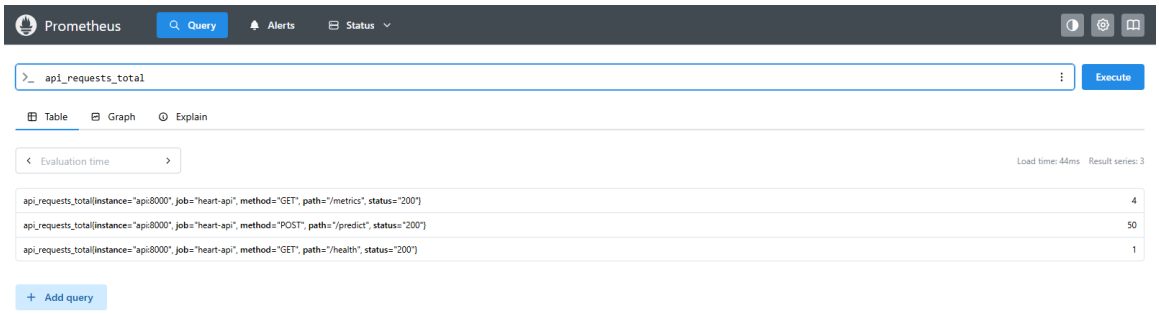


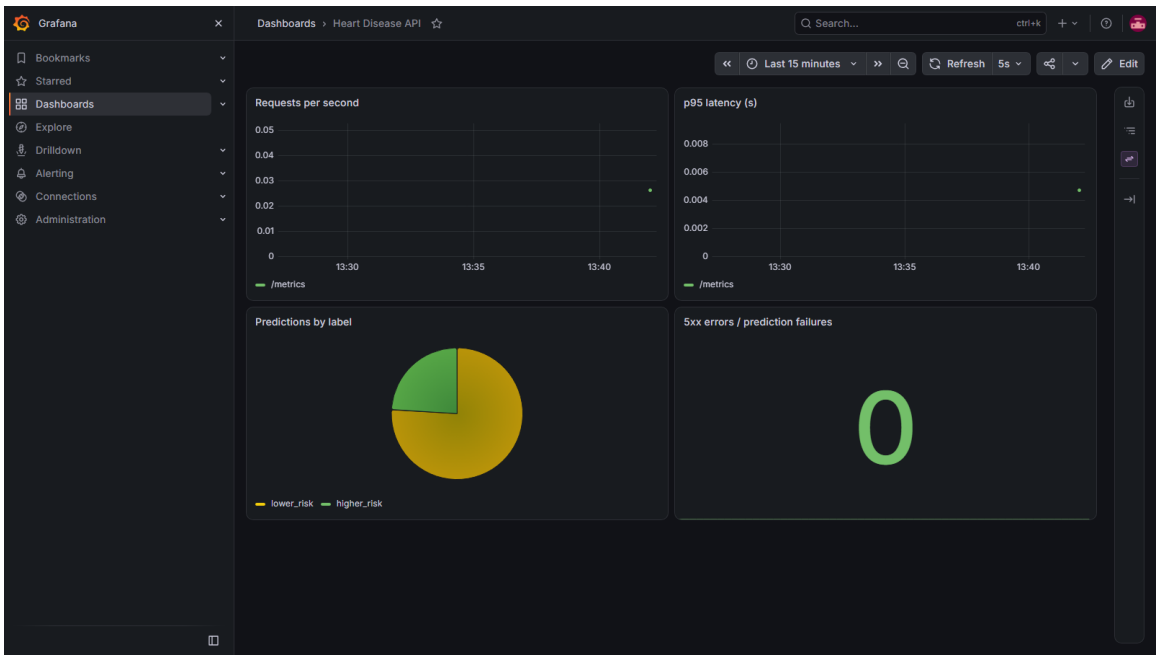


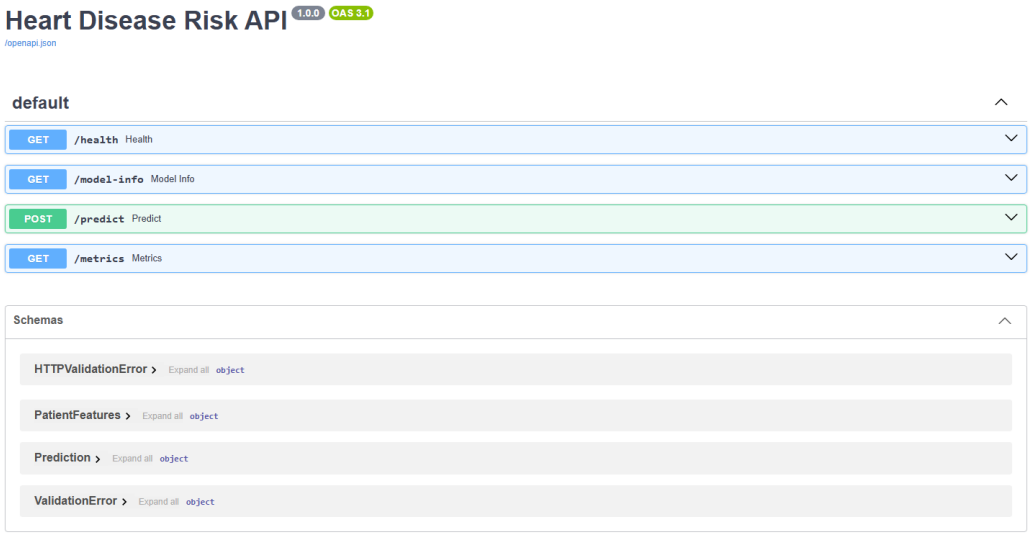
8. Monitoring and logging

Prometheus scrapes the API every five seconds. Its heart-api target was observed up. The provisioned Grafana dashboard displays request rate, p95 latency, predictions by label, and prediction failures.



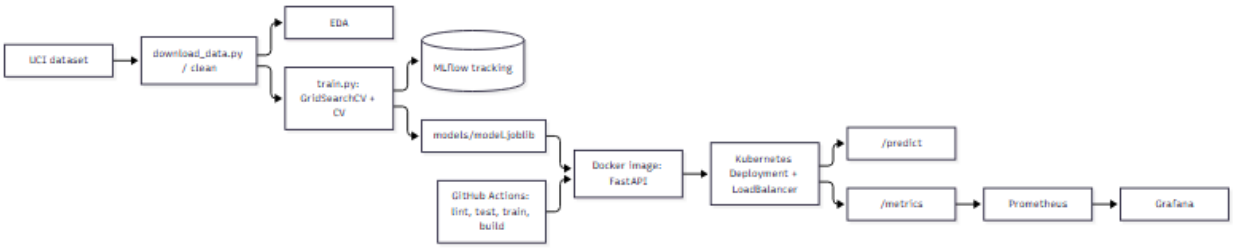

Prometheus
Query
Alerts
Status
api_requests_total
Execute
Table
Graph
Explain
Evaluation time
Load time: 44ms
Result series: 3
api_requests_total{instance="api:8000", job="heart-api", method="GET", path="/metrics", status="200"} 4
api_requests_total{instance="api:8000", job="heart-api", method="POST", path="/predict", status="200"} 50
api_requests_total{instance="api:8000", job="heart-api", method="GET", path="/health", status="200"} 1
Add query


Grafana
Dashboards › Heart Disease API
Search...
ctrl+k
Bookmarks
Starred
Dashboards
Explore
Drilldown
Alerting
Connections
Administration
Last 15 minutes
Refresh
5s
Edit
Requests per second
0.05
0.04
0.03
0.02
0.01
0
13:30
13:35
13:40
/metrics
p95 latency (s)
0.008
0.006
0.004
0.002
0
13:30
13:35
13:40
/metrics
Predictions by label
lower_risk
higher_risk
5xx errors / prediction failures
0



9. Architecture

The diagram follows data acquisition and EDA through model selection, MLflow tracking, FastAPI containerization, Kubernetes deployment, and Prometheus/Grafana monitoring.



10. Local access and submission status

For local use, start the stack with docker compose up --build. The API, Prometheus, and Grafana are exposed on ports 8000, 9090, and 3000. Kubernetes access is available with kubectl port-forward svc/heart-api 8001:80. A video recording was not supplied in the report configuration.